

## Documentație Laborator 5: Testarea algoritmilor de optimizare locală

Drăgoi Mihai-Bogdan

Grupa 10LF242

- Imagine după rularea programului...

```
Instanta TSP incarcata: 131 orase.

Incepere experimente pentru: xqf131_tsp (30 rulari)...
Rularea 30/30...
Experiment finalizat! Rezultate salvate in: rezultate_xqf131_tsp.txt
Instanta Knapsack incarcata: 50 obiecte, Capacitate: 11793

Incepere experimente pentru: s001_knapsack (30 rulari)...
Rularea 30/30...
Experiment finalizat! Rezultate salvate in: rezultate_s001_knapsack.txt
```

- Datele tsp...

[illegible]

- Datele knapsack

[illegible]

## 1. Codificarea Soluției

Clasa „Solution” reprezintă implementarea unei soluții generice, utilizând un vector de numere întregi („std::vector<int> data”). Această structură este adaptabilă în funcție de specificul problemei abordate:

- Pentru problema TSP (Traveling Salesman Problem):
  - Codificarea soluției se bazează pe permutări. Vectorul conține indicii orașelor, indicând ordinea în care acestea sunt vizitate. Primul oraș (index 0) este considerat ca fiind punctul fix de plecare pentru a evita generarea soluțiilor redundante prin rotație.
- Pentru problema Knapsack (Rucsac):
  - Codificarea este de tip binar (un șir de biți). Fiecare element „data[i]” poate lua valoarea 0, indicând că obiectul „i” nu este inclus în rucsac, sau valoarea 1, semnalând că acesta face parte din soluție.

## 2. Generarea Vecinătății

Vecinătatea reprezintă ansamblul soluțiilor care pot fi obținute printr-o singură modificare aplicată soluției actuale:

- Pentru problema TSP (Operatorul Swap):
  - Un vecin este creat prin schimbarea pozițiilor a două orașe din permutare (de exemplu, schimbarea valorilor „data[i]” cu „data[j]”). Această metodă asigură că permutarea rămâne validă.
- Pentru problema Knapsack (Operatorul Bit-Flip):
  - Un vecin este generat prin inversarea stării unui obiect specific (dacă acesta era inclus în rucsac, este eliminat; dacă era exclus, este adăugat).
- Identificatori: Fiecărei mutări i se atribuie un nume unic (cum ar fi "Swap 2-5" sau "Flip 10"). Aceste elemente identificatoare, sub formă de string-uri, sunt esențiale pentru utilizarea eficientă a Listei Tabu, întrucât servesc ca referințe în procesul de căutare.

## 3. Funcția de Evaluare

Funcția *evaluate()* are rolul de a determina calitatea unei soluții în cadrul unor probleme de optimizare, cu obiectivul specific de a minimiza valoarea numerică returnată. Această funcție este utilizată în diverse contexte algoritmice, fiecare având propriile criterii și metode de evaluare.

În cazul *Problemei Comis-Voiajorului (TSP)*, funcția calculează suma distanțelor între orașele vizitate consecutiv. La final, se adaugă distanța dintre ultimul oraș vizitat și orașul de pornire, astfel

închizând circuitul Hamiltonian. Astfel, calitatea unei soluții este proporțională cu lungimea totală a traseului parcurs.

Pentru problema *Knapsack*, valoarea funcției se bazează pe suma profiturilor obiectelor selectate. Deoarece problema inițială presupune maximizarea profitului, abordarea este inversată prin returnarea profitului cu semn negativ (-Profit), transformând astfel problema într-una de minimizare.

Un aspect esențial îl reprezintă și *mecanismul de penalizare*. Atunci când soluția propusă nu respectă constrângerile, spre exemplu, dacă greutatea totală a obiectelor depășește capacitatea maximă a rucsacului, funcția *evaluate()* atribuie soluției o valoare numerică extrem de mare, de ordinul  $1e9$ .

Această penalizare servește drept barieră artificială, descurajând algoritmul în explorarea unor soluții invalide și orientând procesul de optimizare spre respectarea riguroasă a constrângerilor specificate.

#### **4. Strategia de Perturbare**

În cadrul algoritmului *Iterated Local Search (ILS)*, perturbarea joacă un rol esențial în a scoate procesul de căutare din capcana unui optim local.

Pentru aceasta, se implementează un mecanism de *Random Walk* controlat: funcția *perturb(strength)* execută un număr specificat de iterații (determinat de parametrul *strength*) prin apeluri succesive ale funcției *randomNeighbor()*.

Spre deosebire de abordarea *Hill Climbing*, care acceptă doar mutările care îmbunătățesc soluția curentă, perturbarea introduce mutări aleatorii care temporar "deteriorează" soluția. Această metodă permite explorarea unor zone complet diferite ale spațiului de căutare, crescând astfel șansele de a identifica soluții mai bune.

#### **5. Lista Tabu și Construcția**

Algoritmul *Tabu Search* folosește un mecanism de memorie pe termen scurt pentru a preveni repetarea soluțiilor deja analizate, evitând astfel ciclarea și stagnarea în optime locale.

Această strategie sporește eficiența procesului, permițând explorarea de noi variante într-un mod direcționat. Pentru a implementa acest mecanism, se utilizează o structură de date bazată pe „std::map<std::string, int>”, care asociază fiecare mutare efectuată cu un timp sau un număr de iterații în care acea mutare rămâne marcată ca „interzisă”.

Acest interval temporar, cunoscut sub numele de „tabu tenure”, definește perioada în care o mutare este exclusă din explorările ulterioare ale algoritmului. Restricția temporară joacă un rol crucial în diversificarea procesului de optimizare.

Lista Tabu este actualizată continuu pe parcursul fiecărei iterații: durata de interdicție atribuită fiecărei mutări este redusă progresiv prin scăderea valorii întregi asociate din mapă. După ce durata

devine zero, mutarea este eliminată din structura de date, redobândindu-și eligibilitatea pentru analiza ulterioară.

Un aspect esențial al algoritmului este criteriul de aspirație. Acesta permite reconsiderarea unei mutări aflate pe lista Tabu atunci când valoarea adusă de această mutare depășește performanța celei mai bune soluții identificate până în acel moment („bestSoFar”).

Prin intermediul acestui criteriu, algoritmul evită pierderea unui potențial optim global din cauza unei restricții temporare aplicate mutării respective. Această strategie facilitează un echilibru între explorarea de noi soluții și exploatarea celor promițătoare, sporind probabilitatea de a atinge soluția optimă globală.